Coupling the Minkowski loss to flow matching: how it works, what was wrong, and the fix

minkowski-loss project notes (DECISIONS §38–§39)

29 October 2026

Status labels used throughout: [derivation] follows from the stated definitions; [measured] was run in this project (file given); [inference] is our reading, not tested directly; [not verified] means a reference was not opened. All numbers refer to the hourly v4 data (`patches_v4_adv`), the clean flow-matching model `Extremes_20261026_092630/fm_best.pth` and the hourly vanilla backbone `UNet_Ana_20261021_211732`.

Contents

1 The setting in one page **1**

2 The Minkowski functionals and the structural distance **2**

3 How the loss is coupled to flow matching **3**
3.1 The distributional reward 4
3.2 The energy reward 4
3.3 The proximal anchor and the retention term 4

4 Three problems found in the coupling **5**
4.1 The energy fine-tune is mostly more flow-matching training 5
4.2 The anchor is evaluated where the sampler never goes 5
4.3 The dist reward barely moved during training 5

5 Why the reward misses heavy rain **6**

6 Options and the decision **6**

7 The quantile area **7**

8 Inverse-mass weights for perimeter and Euler **8**

9 Checks before any fine-tune **8**

10 Costs, risks and what is not done **9**

1 The setting in one page

Fields and the two spaces. A precipitation field is a vector $f \in \mathbb{R}^N_{\geq 0}$ of hourly accumulations in mm, with $N = 128^2 = 16{,}384$ pixels of $2 \times 2\,\text{km}$. Networks do not work in mm. They work in the *model space*

$$z = \frac{\log(1+f)}{c}, \qquad f = e^{cz} - 1, \qquad c = \log(1 + 613\ \text{mm}) = 6.42, \tag{1}$$

so $z \in [0,1]$. Two consequences of this choice come back later: $\log(1+f) = cz$ exactly, and $\partial f/\partial z = c\,(1+f)$, which is large for heavy pixels.

The flow-matching residual model. A deterministic U-Net (the *backbone*) predicts a smooth field μ in model space from the coarse input X. Flow matching (FM) then learns the *residual*

$$r = \frac{Y-\mu}{\sigma_r}, \qquad \sigma_r = 0.01267, \tag{2}$$

where Y is the target in model space and σ_r makes r roughly unit-scale. FM learns a velocity field $v_\theta(x,t,\mathbf{c})$, with condition $\mathbf{c} = [X,\mu]$, that carries Gaussian noise to residuals. Training draws $x_0 \sim \mathcal{N}(0,I)$, $t \sim U(0,1)$, forms the straight path $x_t = (1-t)x_0 + t\,r$ and regresses the velocity of that path:

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}\,\big\|v_\theta(x_t,t,\mathbf{c}) - (r-x_0)\big\|^2. \tag{3}$$

Sampling integrates $\mathrm{d}x/\mathrm{d}t = v_\theta$ from $t=0$ to 1 with Heun's method, 16 steps of $h = 1/16$:

$$x_{k+1} = x_k + \tfrac{h}{2}\Big[v_\theta(x_k,t_k) + v_\theta\big(x_k + h\,v_\theta(x_k,t_k),\,t_k+h\big)\Big]. \tag{4}$$

The generated field is $f = e^{c\,\mathrm{clamp}(\mu+\sigma_r x_{16},\,0,\,1)} - 1$.

2 The Minkowski functionals and the structural distance

What is measured. For a threshold u, the *excursion set* is the set of pixels that reach it, $E_u = \{i : f_i \ge u\}$. Three numbers describe its geometry: its area $A(u)$, its perimeter $P(u)$ and its Euler characteristic $\chi(u)$ (number of pieces minus number of holes). In two dimensions these are, up to linear combination, the only additive, continuous, motion-invariant functionals of a set [CITE?: Hadwiger's characterisation theorem]. The loss uses 15 thresholds from 0.1 to 79 mm, evenly spaced in $\xi = \log u$.

Discrete, differentiable estimators (`src/losses/minkowski.py`). A hard indicator $\mathbb{1}[f_i \ge u]$ has zero derivative almost everywhere, so the loss uses a *soft mask*

$$s_i(u) = \sigma\!\left(\frac{f_i - u}{\tau_u}\right), \qquad \tau_u = \max(0.1\cdot a\cdot u,\ 10^{-3}), \tag{5}$$

with σ the logistic function and anneal factor $a = 0.05$, so $\tau_u = 0.005\,u$ (0.25 mm at $u = 50$ mm). Then

$$A(u) = \ell^2 \sum_i \mathbb{1}[f_i \ge u] \quad \text{(forward)}, \qquad \frac{\partial A}{\partial f_i} := \ell^2\,\frac{\partial s_i}{\partial f_i} \quad \text{(backward)}, \tag{6}$$

$$P(u) = \ell\Big[w_{\mathrm{ax}} \sum_{\text{axial pairs}} |s_i - s_j| + w_{\mathrm{di}} \sum_{\text{diagonal pairs}} |s_i - s_j|\Big], \tag{7}$$

$$\chi(u) = V - E_x - E_y + F, \tag{8}$$

with $\ell = 2$ km, Cauchy–Crofton weights $w_{\mathrm{ax}} = 0.473$, $w_{\mathrm{di}} = 0.218$, and V, E, F the soft counts of pixels, edges and faces (an edge or face counts with the minimum of its pixels' masks). The area uses a *straight-through estimator*: exact count forward, soft-mask derivative backward.

The curves and the distance. Each field is summarised by three curves over the 15 thresholds,

$$g(f) = \big[\log(1+A),\ \log(1+P),\ \operatorname{sign}(\chi)\log(1+|\chi|)\big] \in \mathbb{R}^{3\times 15}, \tag{9}$$

and two curves are compared by integrating the absolute difference over log-intensity,

$$d(g,h) = \frac{1}{3}\sum_{k=1}^{3} \frac{1}{\xi_{15}-\xi_1}\int_{\xi_1}^{\xi_{15}} \big|g_k(\xi) - h_k(\xi)\big|\,\mathrm{d}\xi, \tag{10}$$

with the integral evaluated by the trapezoid rule on the thresholds. Every decade of intensity gets equal weight in *this sum*. Section 5 shows that equal weight in the sum is not equal weight in the gradient.

3 How the loss is coupled to flow matching

The problem the coupling solves. The FM loss (3) lives on noisy intermediate states x_t, which are not precipitation fields; the functionals only make sense on a finished field. So the structural loss cannot simply be added to (3). The fine-tune instead treats it as a *reward* on generated samples, following DRaFT-K [2]: generate a sample, score it, and back-propagate the score through the sampler into θ.

DRaFT-K with $K = 1$. Back-propagating through all 16 Heun steps would store 32 network evaluations and let the credit for the final sample spread thinly over the whole path. DRaFT-K cuts the graph: the first $16 - K$ steps run with gradients switched off. With $K = 1$,

$$x_{15} = \mathrm{sg}\,\big(15 \text{ Heun steps from } x_0\big), \qquad x_{16} = \Phi_\theta(x_{15}), \qquad \nabla_\theta R = \frac{\partial R}{\partial x_{16}}\,\frac{\partial \Phi_\theta(x_{15})}{\partial \theta}, \tag{11}$$

where sg stops the gradient and Φ_θ is the last Heun step (4). Only the last 1/16 of the path is trained by the reward. [inference] In FM the last step mostly sets fine detail and amplitude, so a $K = 1$ reward acts on the smallest scales, which is where both Minkowski-coupled models later showed excess spectral power. (The coupling check found the $K = 1$ and $K = 2$ gradients aligned, cosine +0.97, [measured], `logs/coupling_v4adv_*.log`.)

One optimiser step. For a batch of $B = 8$ conditions, draw M members per condition, decode each to mm, compute g for each member, and compare with the target curve g^{obs} (precomputed from the observed field). The total loss is

$$L = w_{\mathrm{r}}\, R + w_{\mathrm{p}}\, \mathcal{L}_{\mathrm{prox}} + w_{\mathrm{ret}}\, \mathcal{L}_{\mathrm{FM}}, \tag{12}$$

optimised with AdamW, learning rate 10^{-5}, gradient clipping at norm 1, 500 steps per epoch for 10 epochs.

	FM + dist	FM + energy
reward R	distributional (13)	energy score (15)
members M	4	16
w_{r} / w_{p} / w_{ret}	10^{-3} / 1 / **0**	10^{-3} / 1 / **1**
spread weight λ_s	0.5	(not used)

Table 1: The two fine-tunes (startup lines of `logs/finetune_v4adv_{dist,energy}.log`).

3.1 The distributional reward

$$R_{\text{dist}} = \frac{1}{B}\sum_{b} d\Big(\tfrac{1}{M}\textstyle\sum_m g_{bm},\ g_b^{\text{obs}}\Big) + \lambda_s\, d\Big(\log\big(1+\text{std}_{b,m}\ g_{bm}\big),\ \log\big(1+\text{std}_b\ g_b^{\text{obs}}\big)\Big). \tag{13}$$

The first term asks the *average* member curve of each condition to match the observation. The second asks the spread of the curves to match the spread of the observed curves.

What the spread term actually measures. [derivation] Both standard deviations are pooled over the batch: $\text{std}_{b,m}$ over all $M \times B = 32$ generated curves. By the law of total variance,

$$\text{Var}_{b,m}(g) = \underbrace{\mathbb{E}_b\big[\,\text{Var}_m(g \mid b)\big]}_{\text{ensemble spread}} + \underbrace{\text{Var}_b\left(\mathbb{E}_m[g \mid b]\right)}_{\text{differences between patches}}. \tag{14}$$

Eight different patches differ far more from each other than four members of one patch do, so the second term dominates. As written, the spread term mostly asks the eight patches to differ as much as their targets do, and does little to control the ensemble's own spread.

3.2 The energy reward

$$R_{\text{ES}} = \frac{1}{B}\sum_b \Big[\frac{1}{M}\sum_m d\big(g_{bm}, g_b^{\text{obs}}\big) - \frac{1}{2M(M-1)}\sum_{m\neq m'} d\big(g_{bm}, g_{bm'}\big)\Big]. \tag{15}$$

The first term pulls every member toward the observation; the second rewards members that differ from each other. This is an unbiased estimate of the *energy score* $\text{ES}(P, y) = \mathbb{E}_P\, d(G, y) - \frac{1}{2}\mathbb{E}_P\, d(G, G')$, with G, G' independent draws from the model [CITE?: Gneiting & Raftery 2007, energy score; publisher page not reachable this session].

Why $M(M-1)$ and not M^2. [derivation] For independent members,

$$\mathbb{E}\Big[\frac{1}{M(M-1)}\sum_{m\neq m'} d(G_m, G_{m'})\Big] = \mathbb{E}\, d(G, G').$$

Dividing the same sum by M^2 gives $\frac{M-1}{M}\mathbb{E}\, d(G, G')$ instead: the bonus for spread shrinks, and the minimiser becomes an ensemble that is too narrow, which is the failure the reward is meant to correct.

Why the energy score is the principled choice. A scoring rule is *proper* if its expected value is smallest when the model's distribution equals the true one. For a distance d of negative type, the energy score is proper: the gap between using P and the truth Q is half the energy distance between them, which is non-negative. The trapezoid integral (10) is a weighted L^1 distance between 45-dimensional vectors, and L^1 distances are of negative type, so R_{ES} is proper for the distribution of the curves g [derivation, given the negative-type property]. Whether it is *strictly* proper (the truth is the unique minimiser) with an L^1 distance, we do not know [CITE?: strong negative type of ℓ^1 metrics]. And proper only ever means proper *on the features*: two field distributions with the same distribution of curves score the same.

3.3 The proximal anchor and the retention term

$$\mathcal{L}_{\text{prox}} = \big\|v_\theta(\tilde{x}_t, t, \mathbf{c}) - v_{\text{ref}}(\tilde{x}_t, t, \mathbf{c})\big\|^2, \qquad \tilde{x}_t = (1-t)\,x_0 + t\,\tilde{x}_1, \tag{16}$$

where v_{ref} is the frozen clean model. It should keep the fine-tuned velocity close to the clean one where it matters. The retention term is the ordinary FM loss (3) on the true residual; it supplies a pixel-by-pixel data term, which the reward cannot, because the functionals do not change when a field is shifted or rotated.

4 Three problems found in the coupling

4.1 The energy fine-tune is mostly more flow-matching training

By value, the terms of (12) in the energy run are $w_{\mathrm{r}}R \approx 10^{-3} \times 0.20 = 2 \times 10^{-4}$ and $w_{\mathrm{ret}}\mathcal{L}_{\mathrm{FM}} \approx 0.24$. Values are not gradients, so we measured the gradient of each term separately on four training batches at the clean checkpoint (`scripts/evaluate/grad_norm_terms.py`, `logs/grad_norm_terms_v4adv.txt`) [measured]:

run	reward	anchor	FM retention	reward share	$\cos(\nabla R, \nabla L)$
energy	8.3×10^{-4}	1.2×10^{-4}	0.65	0.13%	−0.01
dist	8.3×10^{-4}	0.7×10^{-4}	0	99%	+0.99

Norms of ∇_θ of $w_{\mathrm{r}}R$, $w_{\mathrm{p}}\mathcal{L}_{\mathrm{prox}}$ and $w_{\mathrm{ret}}\mathcal{L}_{\mathrm{FM}}$, before clipping; share $= \|\nabla(w_{\mathrm{r}}R)\|/\|\nabla L\|$.

Why the absolute size does not matter but the mix does. [derivation] Adam's update is $\Delta\theta = -\eta\,\hat{m}/(\sqrt{\hat{v}} + \epsilon)$, with $\hat{m}, \hat{v}$ running averages of the gradient and its square. Multiplying every gradient by a constant λ multiplies $\hat{m}$ by λ and $\sqrt{\hat{v}}$ by $|\lambda|$, so the step is unchanged (as long as ϵ is negligible). The dist run's tiny total norm (8×10^{-4}, far below the clip at 1) therefore still produces full-size steps, all in the reward's direction. In the energy run the step direction is the FM loss's: the reward is 0.13% of the gradient and orthogonal to the total. The energy fine-tune is, to a good approximation, ten more epochs of FM training at a low learning rate (its `train_fm` fell from 0.2431 to 0.2395). Any difference between FM + energy and clean FM cannot be credited to the reward without the control run with $w_{\mathrm{r}} = 0$.

4.2 The anchor is evaluated where the sampler never goes

The code sets $\tilde{x}_1 = \mathrm{clamp}(f/6.42,\, 0,\, 1)$: the generated field *in mm*, divided by c and saturating at 6.4 mm. But the model's coordinate is the scaled log-residual $r = (Y - \mu)/\sigma_r$, mostly near 0 and of order ± 1, with outliers to $|r| \approx 34$.

Worked example. *A pixel with 10 mm gives* $\tilde{x}_1 = \min(10/6.42, 1) = 1$ *wherever it is. A residual of* 0.05 *in model space (the backbone 0.05 below the target, i.e. a factor* $e^{0.05c} \approx 1.38$ *in* $1 + f$*) gives* $r = 0.05/0.01267 \approx 3.9$.

So $\tilde{x}_t$ blends noise with a binary-looking rain mask, states the ODE does not visit, and the anchor constrains the velocity there, not along the model's real paths [derivation from the code, `reward_finetune.py:348--353`]. In the dist run, which has no retention term, this anchor is the only thing holding the model near the clean one. [inference] That fits the dist model's drift toward small-scale speckle in light rain, which the reward does not penalise.

4.3 The dist reward barely moved during training

Training reward 0.351 → 0.339 over 10 epochs; validation reward 0.361 → 0.350 with swings of ±0.015 between epochs. The coupling check put the per-batch noise at 11% of the mean, so 20 validation batches carry a noise of about ±0.008 on the mean; the total drop is about 1.4 times that [measured]. The model did change (the evaluations show it), but the objective it was trained on improved by an amount near its noise. Either the change is many small steps whose average helps the tail while each batch is too noisy to show it, or it is drift the anchor did not catch. This is why the gain of the distributional fine-tune is not yet attributed.

5 Why the reward misses heavy rain

One pixel, one threshold. [derivation] Through the soft mask (5), pixel i feels threshold u only through

$$\frac{\partial s_i}{\partial f_i} = \frac{1}{\tau_u}\,\sigma'\!\left(\frac{f_i - u}{\tau_u}\right), \qquad \sigma'(a) = \sigma(a)\big(1-\sigma(a)\big) \approx e^{-|a|} \text{ for } |a| \gg 1. \tag{17}$$

The kernel has width $\tau_u = 0.005u$: a pixel feels only the thresholds within a few per cent of its own value.

Worked example. *A predicted core of 17 mm under a target that exceeds 20.2, 31.7 and 50.1 mm. For $u = 20.2$ mm, $\tau = 0.101$ mm and $a = (17-20.2)/0.101 = -31.7$, so $\partial s/\partial f \approx e^{-31.7}/0.101 \approx 1.7\times10^{-13}$ mm^{-1}. For $u = 50.1$, $\tau = 0.25$ mm, $a = -132$ and $e^{-132} \approx 5\times10^{-58}$, which underflows to zero in single precision. The thresholds where the error is largest are the ones that give no gradient.*

All pixels, one threshold: the pixel-mass limit. [derivation] Summing (17) over pixels and writing p for the density of pixel values,

$$\sum_i \frac{\partial s_i}{\partial f_i} = N\int p(f)\,\frac{1}{\tau}\sigma'\!\Big(\frac{f-u}{\tau}\Big)\mathrm{d}f \;\longrightarrow\; N\,p(u) \quad (\tau \to 0), \tag{18}$$

because $\tau^{-1}\sigma'(\cdot/\tau)$ is a probability density of width $\sim\tau$ (it integrates to 1). A threshold's total pull is the number of pixels per mm at u, *whatever τ is*. Two results follow:

- Light rain has most pixels, so it takes most of the gradient: on the clean hourly FM, 73% of $|\partial R/\partial z|$ lands on pixels below 1 mm on the heavy subset and 89% on the random subset (Fig. 2, blue) [measured]. On 15-min data the same limit was measured as "83% of the gradient from thresholds ≤ 0.28 mm/h" (DECISIONS §5).
- Changing the anneal factor changes τ, so it cannot change the shares to leading order. This explains the earlier sweep (DECISIONS §5): anneal 0.05 → 0.40 raised the absolute tail gradient by eleven orders of magnitude while the tail's share stayed at 0.75–0.88%.

The misconception to drop. "Equal weight per decade in the loss means equal weight per decade in training." The distance (10) weights decades equally, but the gradient of each decade's term is multiplied by the number of pixels near it. Equal weight in the objective is not equal pull on the model. The intuition is right for a quantity that is linear in the curve values; it fails here because the curves' sensitivity to the field is itself proportional to pixel mass.

6 Options and the decision

The two failures need different fixes: (1) the gradient grows with pixel mass; (2) a threshold out of the prediction's reach gives nothing. All options keep $K = 1$.

A. Inverse-mass weights. Weight each threshold's term by $(n_j+1)^{-\alpha}$, where n_j is the target's number of pixels above u_j. Fixes (1), not (2). A few lines.

B. Hard forward, heavy-tailed backward. Keep exact counts forward and replace σ' in the backward pass by a heavy-tailed kernel (e.g. Cauchy) of width $\sim 0.3u$ at high thresholds. Fixes (2) with A for (1). Needs the same straight-through treatment for P and χ, adds a width hyperparameter, and a surrogate gradient can point where the true objective does not, which is how gaming starts.

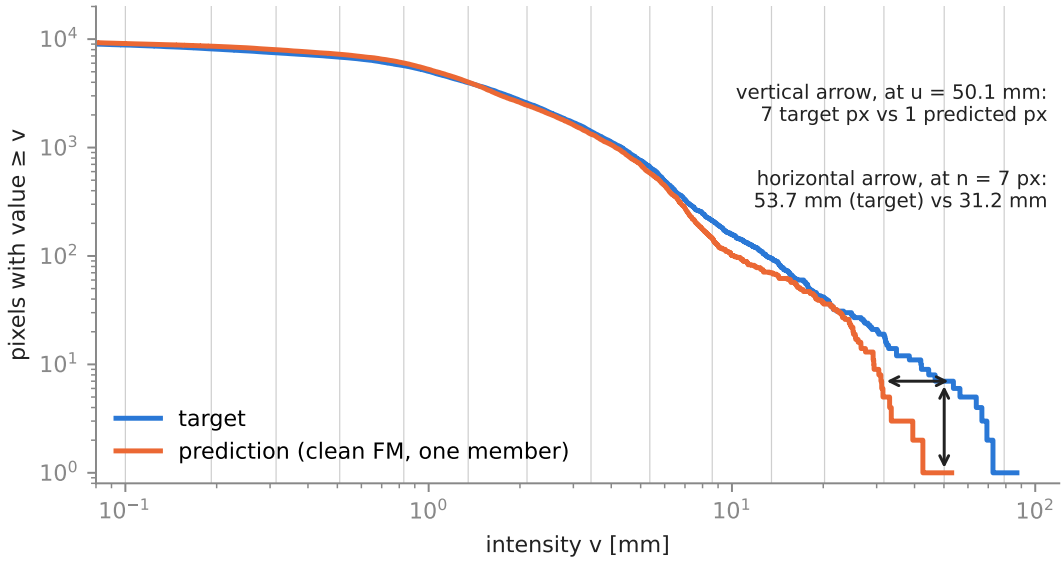


Figure 1: Survival curves of a real heavy validation patch (patch 979, target maximum 87 mm) and one clean-FM member. Vertically, at $u = 50.1$ mm, the target has 7 pixels above the threshold and the prediction 1; the vertical term compares $\log(1 + A)$ values and its gradient needs a predicted pixel near 50.1 mm. Horizontally, at the same count $n = 7$, the target reaches 53.7 mm and the prediction 31.2 mm; this difference has a gradient on the predicted pixel of rank 7 wherever its value is.

C. Quantile form of the area. Compare the area curve horizontally instead of vertically (next section). Removes (2) exactly for the area channel, with no new hyperparameter. Needs A for P and χ.

Decision (researcher, 2026-10-29): C for the area, A for perimeter and Euler. In the code this is `REWARD_FORM: quantile_mass`; the original stays available as `REWARD_FORM: threshold`, the default.

7 The quantile area

Same curve, read along the other axis. The area curve is a *survival function*: $S_f(v) = \#\{i : f_i \geq v\}$, so $A(u) = \ell^2 S_f(u)$. Plot it with intensity on the horizontal axis and pixel count on the vertical axis (Fig. 1). The original loss compares the two curves *vertically*: at a fixed intensity u_j, how many pixels each field has above it. One can equally compare them *horizontally*: at a fixed count n, which intensity each field reaches. The intensity at which a field has exactly n pixels above it is its n-th largest value, written $f_{(n)}$.

The new area channel. For condition b and threshold u_j, let $n_j = \#\{i : y_i \geq u_j\}$ be the target's count. The area entry of the curve becomes

$$q_j(f) = \log\left(1 + f_{(n_j)}\right), \qquad q_j(y) = \log\left(1 + y_{(n_j)}\right), \tag{19}$$

masked (weight 0) where $n_j = 0$, i.e. above the target maximum. Since $y_{(n_j)}$ is the smallest target value that is still $\geq u_j$, the target side sits just above each threshold.

Its gradient. [derivation] In model space $\log(1 + f) = cz$, so $q_j(f) = c\, z_{(n_j)}$ and

$$\frac{\partial}{\partial z_i}\left|q_j(f) - q_j(y)\right| = \pm c \quad \text{for the pixel of rank } n_j, \quad 0 \text{ otherwise.} \tag{20}$$

Every populated threshold moves exactly one pixel per member, with the same magnitude c, whatever that pixel's value. (Sorting is differentiable in the sense needed: the sorted values are a permutation of the inputs, and the gradient flows to the selected element.) In Fig. 1 the term at $n = 7$ pushes the predicted 31.2 mm pixel up toward 53.7 mm, where the vertical term gives nothing.

Link to Wasserstein distances, and where it breaks. For two sets of N values, the 1-D Wasserstein distance is $W_1 = \frac{1}{N}\sum_n |f_{(n)} - y_{(n)}|$: a horizontal comparison of the two distributions at every rank. Arjovsky et al. make the corresponding argument for distributions: a horizontal (transport) distance gives a usable gradient where a vertical comparison saturates because the supports do not overlap [1]. Our area term is a horizontal comparison at 15 ranks only, in log-intensity, per patch. The analogy breaks in two places: only ≤ 15 pixels per member get a gradient (not all N), and the comparison is per patch, not between distributions of patches.

8 Inverse-mass weights for perimeter and Euler

Perimeter and Euler stay vertical (they have no simple horizontal form) and are weighted per threshold:

$$w_j = \frac{(n_j + 1)^{-\alpha}}{\frac{1}{15}\sum_{j'}(n_{j'} + 1)^{-\alpha}}, \qquad \alpha = 1, \tag{21}$$

so the weights average 1. Thresholds above the target maximum ($n_j = 0$) take the weight of the highest populated threshold, so that overshoot is penalised as strongly as the top of the target, not infinitely. The weighted distance replaces $|g_k - h_k|$ by $w\,|g_k - h_k|$ inside (10).

Why $\alpha = 1$. [derivation under an assumption] The pull of threshold u is the number of pixels near it. Suppose the survival curve is locally a power law, $S(v) \propto v^{-k}$. The number of pixels per unit of log-intensity at u is then $-\mathrm{d}S/\mathrm{d}\log v = k\,S(u) \propto n_j$. Weighting by $1/n_j$ cancels it. The assumption fails where the survival curve bends (the flat top below 1 mm in Fig. 1), and the perimeter and Euler sensitivities depend on boundary pixels rather than on all pixels near u, so we checked $\alpha \in \{0, 0.5, 1\}$ directly (below) and $\alpha = 1$ gave the most even spread across intensity bins.

Properness is kept. [derivation] The ranks n_j and the weights w_j depend only on the target. For each condition the reward therefore compares fixed feature vectors with a fixed weighted L^1 distance (non-negative weights keep the negative-type property; zero weights make it a pseudo-metric on the features). The energy score stays proper for the distribution of these features. If the weights depended on the prediction, this would fail.

9 Checks before any fine-tune

Unit tests (`tests/test_reward_form.py`, 13 tests, all pass; on the previous code the file fails at import) [measured]:

- a 17 mm predicted core under an 80 mm target: the original form gives the core no gradient, the new form pulls it up (negative $\partial R/\partial f$), in all three modes;
- unit weights reproduce the original reward exactly, in all three modes;
- the area feature equals $\log(1 + f_{(n_j)})$ and is masked where $n_j = 0$; an exact prediction scores 0;
- weights depend on the target only, average 1, grow as n_j falls, and are capped above the target maximum.

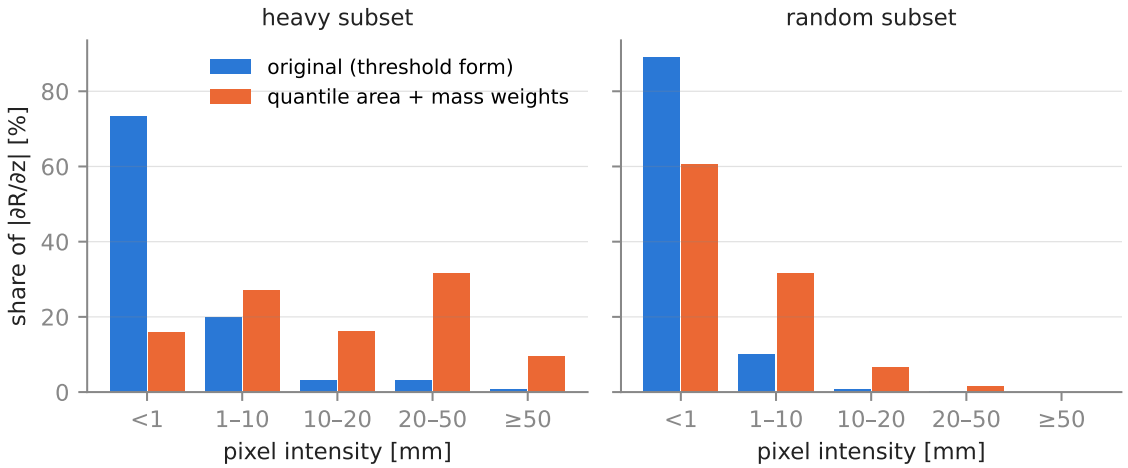


Figure 2: Share of the reward's model-space gradient $|\partial R/\partial z|$ landing on generated pixels of each intensity bin, original form vs quantile area + mass weights ($\alpha = 1$). Clean FM, energy mode, $M = 4$.

Where the gradient lands (`scripts/evaluate/reward_threshold_share.py`, energy mode, $M = 4$, 4 batches of 8, `logs/reward_threshold_share_v4adv.txt`) [measured]. Fig. 2: on the heavy subset the share of the gradient on pixels ≥ 20 mm goes from 3.7% to 40.9%, while pixels below 1 mm keep 15.9%; on the random subset, which has few heavy pixels, the shift is smaller and in the same direction. With $\alpha = 0$ (quantile area alone) almost nothing changes, because the area channel carried only 11% of the gradient; the Euler term at 0.1 mm alone carried 17% (heavy) to 28% (random).

Coupling checks (`scripts/evaluate/diagnose_coupling.py`, one heavy batch, fixed noise, 40 reward-only steps; `logs/coupling_v4adv_*_qm.log`) [measured]:

reward	reward drop	noise floor (2 sd)	sampling noise / mean
distributional, original	−28.1%	0.071	11.4%
distributional, new	−31.6%	0.108	18.9%
energy, original	−32.5%	0.027	7.2%
energy, new	−35.1%	0.024	8.4%

Both new rewards are drivable: the gradient reaches the weights in a usable form, and the $K = 1$ and $K = 2$ gradients still agree (cosine +0.98 dist, +0.97 energy).

Gradient split per term for the new form [measured]: with the new form the reward's gradient norm is 5.0×10^{-4} in both modes (was 8.3×10^{-4}). In the dist configuration it is 96% of the total (cos +0.97). In the energy configuration it is **0.07%** of the total (cos −0.03): the retention term still sets the direction of every step. A fine-tune of the energy configuration with the new form would therefore again be continued FM training. Before running one, the reward and retention weights must be rebalanced, e.g. by matching their gradient norms (the gradient-norm rule of DECISIONS §9), which here means raising w_{r} by roughly three orders of magnitude or lowering w_{ret} accordingly.

10 Costs, risks and what is not done

- **Noisier reward.** Giving rare thresholds weight raises the reward's sampling noise (distributional: 11.4% → 18.9% of the mean). Training-time changes will be harder to see on 20

validation batches.

- **Artefacts get more weight.** The top populated threshold may hold a handful of pixels, and the top of the distribution contains radar artefacts. The new reward pulls harder toward them. Look at images of anything it produces.
- **Sparse area gradient.** At most 15 pixels per member per step from the area term. The overall field gradient was already concentrated before (100% of its mass on the top 1% of pixels in both forms).
- **Properness only on the features.** The reward cannot see anything the 45 numbers per field miss, including position: the functionals carry no positional information, so placement must come from a data term.
- **Not a result yet.** No fine-tune has run with the new form. Before one, two items in EXPERIMENTS §5: fix the proximal anchor so it is evaluated on the model's own coordinates, and run the $w_{\mathrm{r}} = 0$ control to attribute the energy run. For the energy configuration, also rebalance w_{r} against w_{ret}: as configured, its reward is 0.07% of the gradient with either form.
- **Where the picture stops holding.** The pixel-mass argument assumes p varies slowly over τ; the $\alpha = 1$ argument assumes a locally power-law survival curve; and all gradient measurements are at the clean checkpoint, before any fine-tuning has moved the model.

Other items added to the to-do list (EXPERIMENTS §5). Plot units on hourly data: the colourbars, axis labels and titles in `src/evaluation/plotting.py` say $\mathrm{mm\,h^{-1}}$; from v4 the target is a 1-h accumulation and they should say mm (the 15-min figures are correct as they are).

References

[1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein GAN, 2017. URL https://arxiv.org/abs/1701.07875.

[2] Kevin Clark, Paul Vicol, Kevin Swersky, and David J. Fleet. Directly fine-tuning diffusion models on differentiable rewards. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2309.17400.